

TRƯỜNG: ĐH Công thương TP HCM KHOA: CÔNG NGHỆ THÔNG TIN BỘ MÔN: KHD&TTNT MH: LẬP TRÌNH PYTHON	BUỔI 10. Tkinter – Layout và một số thành phần nâng cao	
--	--	---

A. MỤC TIÊU

- Thiết kế giao diện với tkinter

B. DỤNG CỤ - THIẾT BỊ THÍ NGHIỆM CHO MỘT SV

STT	Chủng loại – Quy cách vật tư	Số lượng	Đơn vị	Ghi chú
1	Computer	1	1	

C. NỘI DUNG THỰC HÀNH

1. Layout Management

1.1. Layout Management là gì?

Layout Management là quá trình tổ chức và sắp xếp các thành phần trên giao diện của chương trình. Trong tkinter, có ba phương thức chính được sử dụng cho Layout Management:

- Pack
- Grid
- Place

1.2. Phương thức Pack

Phương thức Pack trong Tkinter được sử dụng để đặt các widget trên giao diện theo hướng ngang hoặc dọc. Nó được ưa chuộng vì tính đơn giản và dễ sử dụng, đặc biệt là cho các ứng dụng đơn giản.

1.2.1. Các Tùy Chọn

side:

- "top": đặt widget ở phía trên.
- "bottom": đặt widget ở phía dưới.
- "left": đặt widget ở phía trái.
- "right": đặt widget ở phía phải.

fill:

- "x": điền widget theo chiều ngang.
- "y": điền widget theo chiều dọc.
- "both": điền widget theo cả hai chiều.

expand:

- True/False: mở rộng widget để điền vào khoảng trống.

1.2.2. Ví dụ

```
import tkinter as tk

root = tk.Tk()
root.title("Sử Dụng Phương Thức Pack")

# Tạo các widget
label1 = tk.Label(root, text="Label 1", bg="red", fg="white")
label2 = tk.Label(root, text="Label 2", bg="green", fg="white")
label3 = tk.Label(root, text="Label 3", bg="blue", fg="white")

# Sắp xếp các widget bằng Pack
label1.pack(side="top", fill="x")
label2.pack(side="left", fill="y")
label3.pack(side="right", fill="both", expand=True)

root.mainloop()
```

Kết quả



Nếu mở rộng cửa sổ, ta có kết quả



1.3. Phương thức grid

Phương thức Grid trong Tkinter được sử dụng để đặt các widget trên giao diện theo lưới ô. Nó cho phép chúng ta sắp xếp các widget theo hàng và cột, tạo ra các giao diện có cấu trúc lưới phức tạp.

1.3.1. Các Tùy Chọn

- row: số hàng.
- column: số cột.
- sticky:
 - "n", "s", "e", "w": chỉ định cách widget chạy nếu có không gian nhiều hơn kích thước widget.
 - Kết hợp của các giá trị trên: ví dụ, "nesw" để căn lề widget theo cả bốn hướng.

1.3.2. Ví Dụ

```
import tkinter as tk

root = tk.Tk()
root.title("Sử Dụng Phương Thức Grid")

# Tạo các widget
label1 = tk.Label(root, text="Label 1", bg="red", fg="white")
label2 = tk.Label(root, text="Label 2", bg="green", fg="white")
label3 = tk.Label(root, text="Label 3", bg="blue", fg="white")
```

```
# Sắp xếp các widget bằng Grid
label1.grid(row=0, column=0, sticky="ns")
label2.grid(row=1, column=0, sticky="ew")
label3.grid(row=0, column=1, rowspan=2, sticky="nsew")

# Đặt trọng số cho cột và hàng để widget mở rộng
root.columnconfigure(0, weight=1)
root.columnconfigure(1, weight=1)
root.rowconfigure(0, weight=1)
root.rowconfigure(1, weight=1)

root.mainloop()
```

Kết quả



Nếu mở rộng cửa sổ, ta có kết quả



1.3.3. *rowconfigure* và *columnconfigure*

Phương thức *columnconfigure* và *rowconfigure* trong Tkinter được sử dụng để cấu hình các cột và hàng của lưới trong phương thức Grid. Chúng cho phép điều chỉnh kích thước và hành vi mở rộng của các cột và hàng trong lưới.

```
import tkinter as tk
```

```

root = tk.Tk()
root.title("Sử Dụng columnconfigure và rowconfigure")

label1 = tk.Label(root, text="Label 1", bg="red", fg="white")
label2 = tk.Label(root, text="Label 2", bg="green", fg="white")
label3 = tk.Label(root, text="Label 3", bg="blue", fg="white")

label1.grid(row=0, column=0, sticky="nsew")
label2.grid(row=1, column=0, sticky="nsew")
label3.grid(row=0, column=1, rowspan=2, sticky="nsew")

# Cấu hình cột và hàng để widget mở rộng
root.columnconfigure(0, weight=1)
root.columnconfigure(1, weight=2)
root.rowconfigure(0, weight=1)
root.rowconfigure(1, weight=1)

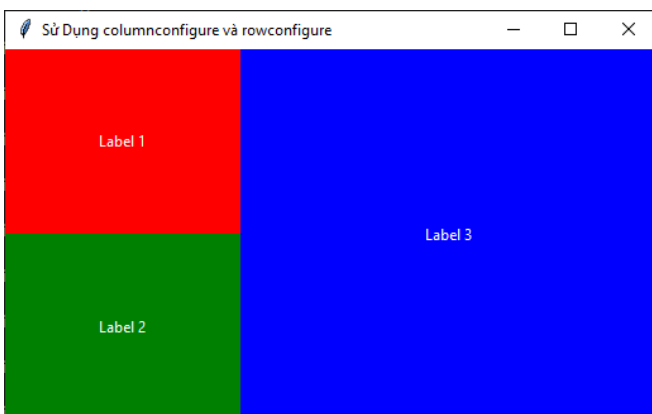
root.mainloop()

```

Kết quả



Nếu mở rộng cửa sổ, ta có kết quả



Cột chứa label3 sẽ rộng hơn cột label1 và label2 do weight = 2.

1.4. Phương thức Place

Phương thức Place trong Tkinter được sử dụng để đặt các widget trên giao diện bằng cách chỉ định các tọa độ tuyệt đối. Nó cung cấp sự linh hoạt tuyệt đối trong việc định vị các widget và làm cho chúng ta có thể đặt các widget ở bất kỳ vị trí nào trên cửa sổ.

1.4.1. Ví dụ 1

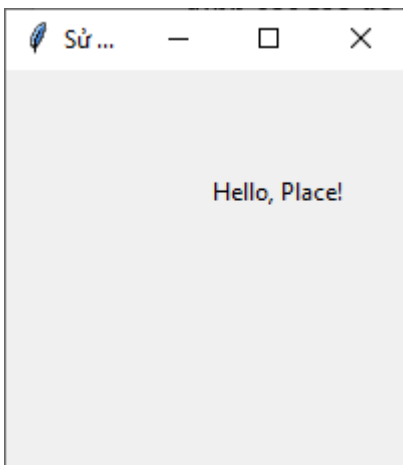
```
import tkinter as tk

root = tk.Tk()
root.title("Sử Dụng Phương thức Place")

# Tạo label
label = tk.Label(root, text="Hello, Place!")
label.place(x=100, y=50) # Đặt label ở tọa độ (100, 50)

root.mainloop()
```

Kết quả



1.4.2. Ví dụ 2

```
import tkinter as tk

root = tk.Tk()
root.title("Sử Dụng Phương thức Place")
```

```
# Tạo các label
label1 = tk.Label(root, text="Label 1", bg="red", fg="white")
label2 = tk.Label(root, text="Label 2", bg="green", fg="white")
label3 = tk.Label(root, text="Label 3", bg="blue", fg="white")

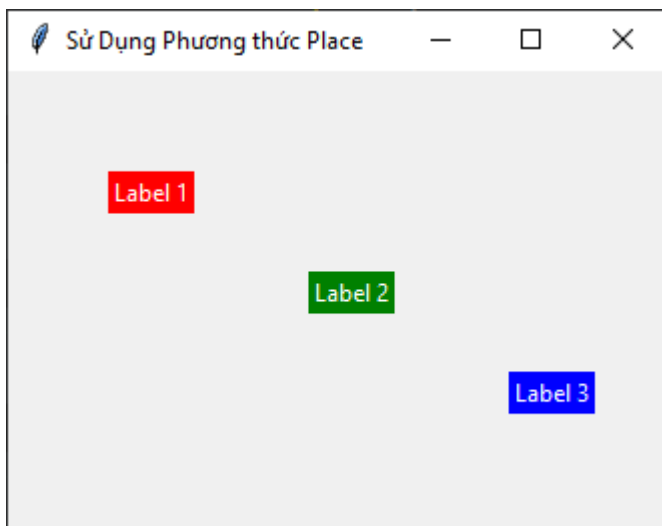
# Đặt label1 ở tọa độ (50, 50)
label1.place(x=50, y=50)

# Đặt label2 ở tọa độ (150, 100)
label2.place(x=150, y=100)

# Đặt label3 ở tọa độ (250, 150)
label3.place(x=250, y=150)

root.mainloop()
```

Kết quả



1.4.3. Ưu điểm và khuyết điểm

Ưu điểm của Phương thức Place:

- Linh hoạt: Cho phép bạn đặt các widget ở bất kỳ vị trí nào trên cửa sổ.
- Chính xác: Dễ dàng định vị các widget với độ chính xác cao thông qua tọa độ tuyệt đối.

- Đa dạng: Cho phép bạn đặt các widget lên nhau hoặc chồng lên nhau, tạo ra giao diện phức tạp.

Nhược điểm của Phương thức Place:

- Không Tự Điều Chỉnh: Cần phải tự điều chỉnh lại vị trí của các widget khi cửa sổ thay đổi kích thước.
- Khó Quản Lý: Có thể trở nên khó khăn khi có quá nhiều widget và định vị chúng bằng cách Place.

1.4.4. Các tùy chọn

- x và y: Tọa độ tuyệt đối của góc trên bên trái của widget, tính bằng pixel từ góc trên bên trái của cửa sổ.
- width và height: Chiều rộng và chiều cao của widget, tính bằng pixel.
- anchor: Điểm neo của widget, quyết định vị trí mà widget sẽ được neo tới tọa độ x và y. Các giá trị có thể là "n", "ne", "e", "se", "s", "sw", "w", "nw", "center" hoặc kết hợp của chúng. Mặc định là "nw" (góc trên bên trái).
- relx và rely: Tọa độ tương đối của widget trong phạm vi từ 0.0 đến 1.0, đại diện cho phần trăm vị trí của widget trên cửa sổ theo chiều ngang (relx) và chiều dọc (rely).
- relwidth và relheight: Chiều rộng và chiều cao của widget theo phần trăm của cửa sổ. Giá trị của chúng cũng nằm trong phạm vi từ 0.0 đến 1.0.
- bordermode: Chỉ định cách tính toán độ dày của đường viền khi thiết lập các thuộc tính relwidth và relheight. Giá trị có thể là "inside" (mặc định) hoặc "outside".
- in_ (hoặc in): Widget hoặc master widget mà widget được đặt bên trong.

1.4.5. Ví dụ 3

```
import tkinter as tk

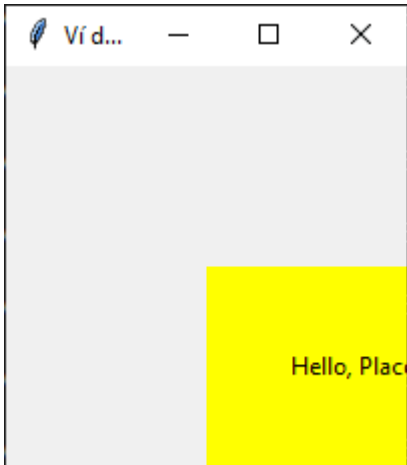
root = tk.Tk()
root.title("Ví dụ Sử Dụng Phương thức Place với Các Tùy Chọn")

# Tạo label
label = tk.Label(root, text="Hello, Place!", bg="yellow",
fg="black")

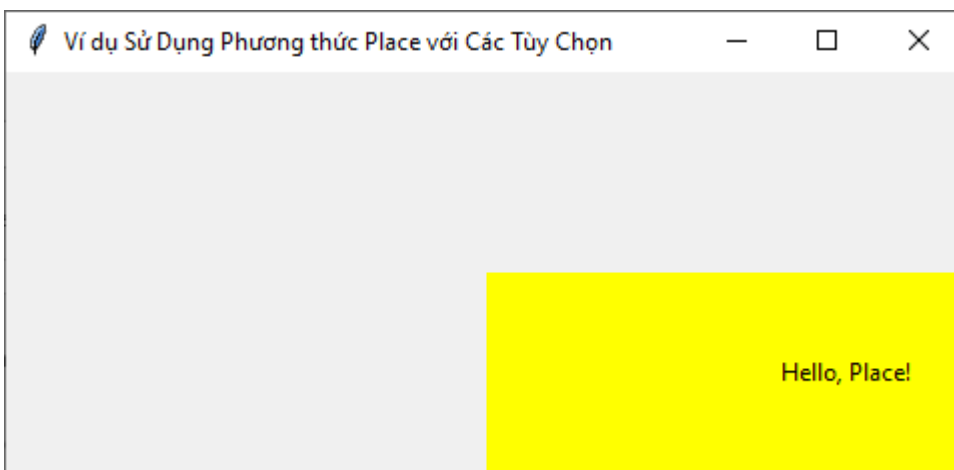
# Sử dụng các tùy chọn place
label.place(anchor="nw", relx=0.5, rely=0.5, relwidth=0.75,
relheight=0.5, bordermode="inside")

root.mainloop()
```


Kết quả



Nếu mở rộng cửa sổ



Trong ví dụ này, chúng ta đã sử dụng các tùy chọn sau:

- `anchor="nw"`: Widget được neo tại góc trên bên trái.
- `relx=0.5` và `rely=0.5`: Đặt widget tại vị trí giữa của cửa sổ theo chiều ngang và dọc.
- `relwidth=0.75` và `relheight=0.5`: Thiết lập chiều rộng là 75% và chiều cao là 50% của cửa sổ.
- `bordermode="inside"`: Tính toán độ dày của đường viền bên trong kích thước của widget.

1.4.6. Ví dụ 4

```
import tkinter as tk

root = tk.Tk()
root.title("Ví dụ Sử Dụng Phương thức Place với Nhiều Label")

# Tạo các label
label1 = tk.Label(root, text="Label 1", bg="red", fg="white")
```

```

label2 = tk.Label(root, text="Label 2", bg="green", fg="white")
label3 = tk.Label(root, text="Label 3", bg="blue", fg="white")

# Sử dụng các thuộc tính place
label1.place(x=50, y=50, width=100, height=50, anchor="nw")
label2.place(relx=0.5, rely=0.5, relwidth=0.6, relheight=0.3,
anchor="center")
label3.place(x=200, y=200, width=150, height=100, anchor="se")

root.mainloop()

```

Kết quả

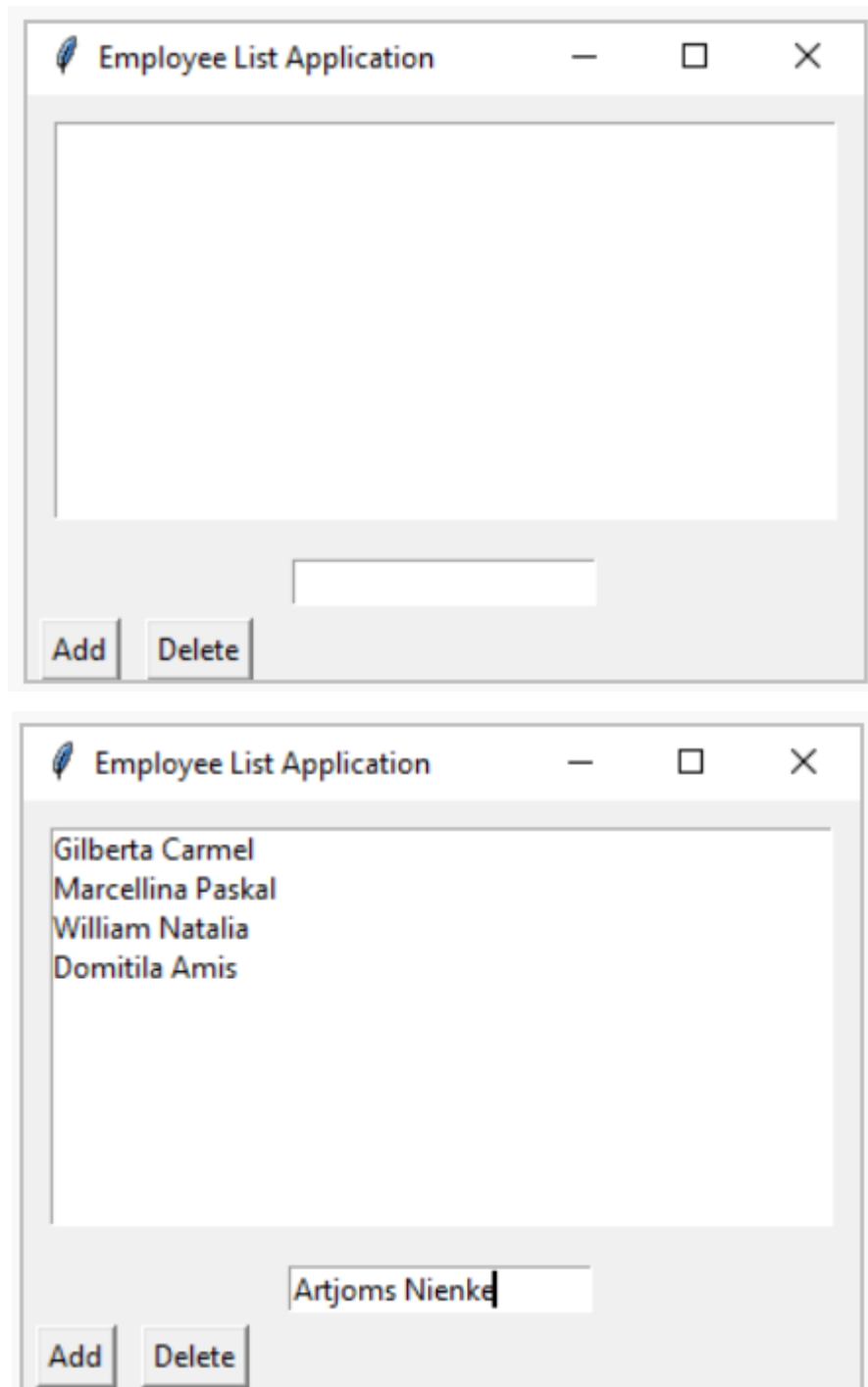


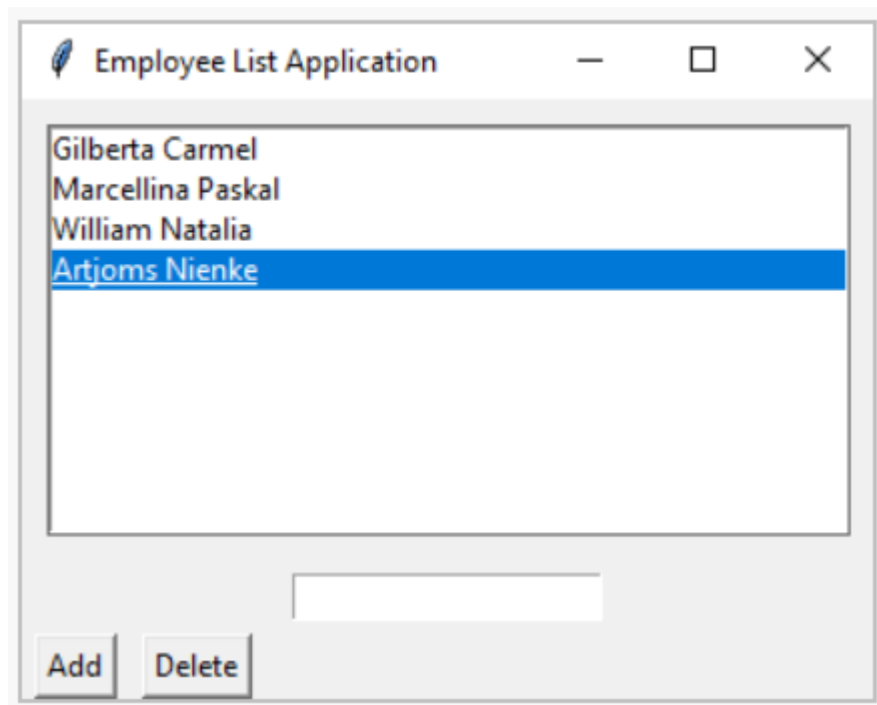
BÀI TẬP TRÊN LỚP PHẦN 1

- Viết và cài đặt lại những đoạn code trong bài học này.
- Viết chương trình python xây dựng ứng dụng máy tính điện tử đơn giản. Các button được sắp xếp sử dụng grid.

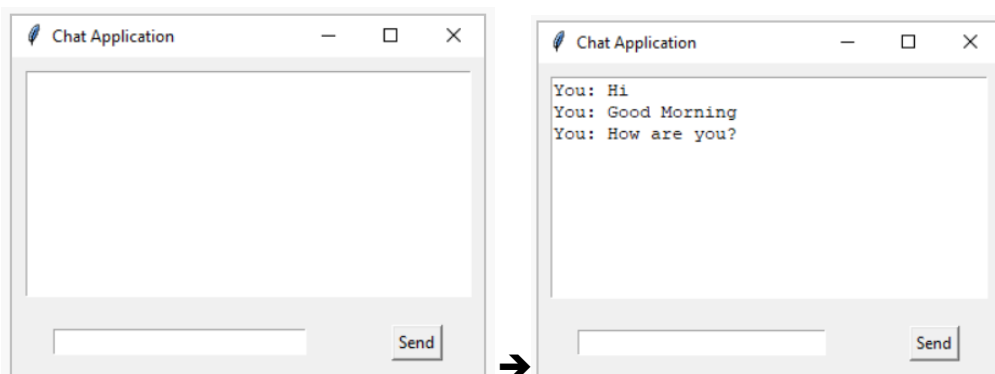


3. Viết một chương trình Python để xây dựng một ứng dụng danh sách nhân viên với một widget Listbox để hiển thị tên. Sử dụng Pack để căn chỉnh Listbox và các nút.

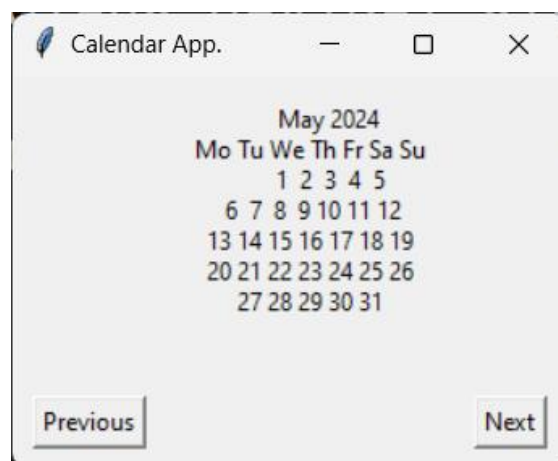




4. Viết một chương trình Python để thiết kế giao diện ứng dụng chat với lịch sử tin nhắn và một trường Entry. Tổ chức các phần tử này bằng cách sử dụng Grid. (Lưu ý: chỉ thiết kế giao diện, không cần thiết kế chức năng chat qua mạng)



5. Viết một chương trình Python để tạo một ứng dụng lịch cơ bản với các nhãn cho các ngày trong tuần và các ngày trong tháng. Sử dụng quản lý hình học Grid để sắp xếp các nhãn.



6. Viết một chương trình Python triển khai một hệ thống đặt chỗ ghế cho một phòng hội trường, sắp xếp các nút ghế theo grid.



Khi một ghế được đặt, đổi màu nút



2. Menu trong tkinter

Menu là một phần quan trọng trong bất kỳ ứng dụng đồ họa nào, bao gồm cả ứng dụng desktop và ứng dụng di động. Trong Tkinter, Menu là một thành phần giao diện người dùng cho phép người dùng tương tác với các chức năng và tùy chọn của ứng dụng.

2.1. Tại sao cần Menu ?

- Dễ dàng sử dụng: Menu cung cấp một giao diện trực quan cho người dùng để truy cập các chức năng và tùy chọn của ứng dụng một cách dễ dàng. Người dùng có thể chọn các lệnh từ menu thay vì phải nhớ các phím tắt hoặc câu lệnh dòng lệnh.
- Tổ chức chức năng: Menu giúp tổ chức chức năng của ứng dụng thành các nhóm logic như "File", "Edit", "View", "Help",... giúp người dùng dễ dàng tìm kiếm và sử dụng các chức năng một cách hiệu quả.
- Tiết kiệm không gian: Menu cho phép ẩn đi một số chức năng ít được sử dụng trong một menu con hoặc trong một menu thả xuống, giúp tiết kiệm không gian trên giao diện người dùng.

- Tăng tính tương tác: Menu cung cấp một cách tương tác tự nhiên cho người dùng. Họ có thể duyệt qua các lựa chọn, xem trước các tùy chọn và chọn chức năng một cách dễ dàng thông qua các menu thả xuống và các menu con.

2.2. Các bước tạo menu

- Tạo cửa sổ gốc (root): Bước đầu tiên là tạo một cửa sổ gốc sử dụng `tkinter.Tk()`.
- Tạo MenuBar: Tạo một đối tượng MenuBar sử dụng `tkinter.Menu()`.
- Thêm các menu vào MenuBar: Sử dụng phương thức `add_cascade()` hoặc `add_command()` để thêm các menu vào MenuBar. Menu có thể là các menu con hoặc các lệnh trực tiếp.
- Thêm các lệnh vào các menu: Sử dụng phương thức `add_command()` để thêm các lệnh vào menu. Mỗi lệnh có thể có một hàm gọi lại để xử lý khi lệnh được nhấp.
- Kích hoạt menu: Gọi phương thức `config(menu=menubar)` trên cửa sổ gốc để kích hoạt MenuBar.

2.3. Ví dụ

```
import tkinter as tk

def hello():
    print("Hello!")

def quit_app():
    root.quit()

# Bước 1: Tạo cửa sổ gốc
root = tk.Tk()
root.title("Menu Demo")

# Bước 2: Tạo MenuBar
menubar = tk.Menu(root)

# Bước 3: Thêm các menu vào MenuBar
file_menu = tk.Menu(menubar, tearoff=0)
menubar.add_cascade(label="File", menu=file_menu)
file_menu.add_command(label="New", command=hello)
```

```

file_menu.add_command(label="Open", command=hello)
file_menu.add_separator()
file_menu.add_command(label="Exit", command=quit_app)

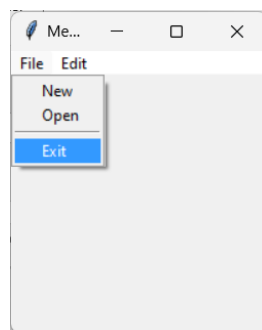
edit_menu = tk.Menu(menubar, tearoff=0)
menubar.add_cascade(label="Edit", menu=edit_menu)
edit_menu.add_command(label="Cut", command=hello)
edit_menu.add_command(label="Copy", command=hello)
edit_menu.add_command(label="Paste", command=hello)

# Bước 5: Kích hoạt MenuBar
root.config(menu=menubar)

# Hiển thị cửa sổ
root.mainloop()

```

Kết quả



3. Colorchooser

Colorchooser trong Tkinter là một thành phần giao diện người dùng cho phép người dùng chọn màu sắc từ một bảng màu hoặc nhập mã màu để sử dụng trong các ứng dụng đồ họa. Thành phần này cung cấp một cách tiện lợi để chọn màu sắc mà không cần phải nhớ mã màu hoặc sử dụng các công cụ bên ngoài.

3.1. Các tùy chọn

Colorchooser trong Tkinter cung cấp một số tùy chọn để tùy chỉnh và điều chỉnh chọn màu sắc. Một số tùy chọn phổ biến bao gồm:

- Initial color (màu ban đầu): Màu sắc xuất phát được hiển thị khi colorchooser được mở lần đầu tiên.
- Title (tiêu đề): Tiêu đề của cửa sổ colorchooser.
- RGB (Red, Green, Blue): Các giá trị màu sắc của màu được chọn có thể được trả về dưới dạng RGB.
- HEX (Hexadecimal): Mã màu Hexadecimal của màu được chọn cũng có thể được trả về.

3.2. Ví dụ:

```
import tkinter as tk
from tkinter import colorchooser

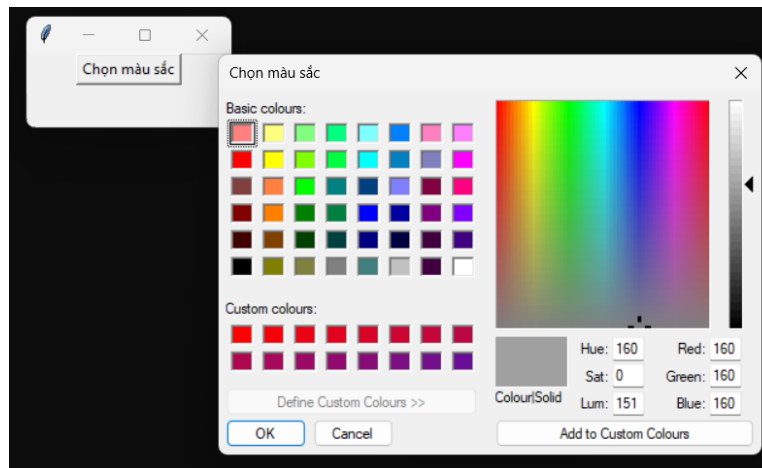
def choose_color():
    color = colorchooser.askcolor(title="Chọn màu sắc")
    print("Màu được chọn:", color)

root = tk.Tk()
root.title("Ví dụ Colorchooser")

button = tk.Button(root, text="Chọn màu sắc",
command=choose_color)
button.pack()

root.mainloop()
```

Kết quả



4. Date Entry

Date Entry trong Tkinter là một thành phần giao diện người dùng cho phép người dùng chọn ngày, tháng và năm từ một cửa sổ lịch. Thành phần này cung cấp một cách tiện lợi để chọn ngày tháng năm trong các ứng dụng yêu cầu nhập ngày tháng, như lịch làm việc, đặt hẹn, và các ứng dụng quản lý sự kiện khác.

4.1. Các tùy chọn

Date Entry trong Tkinter có một số tùy chọn để tùy chỉnh và điều chỉnh cách hiển thị và cách người dùng có thể nhập ngày tháng năm. Một số tùy chọn phổ biến bao gồm:

Format (định dạng): Định dạng ngày tháng năm hiển thị trong Date Entry, như dd/mm/yyyy hoặc mm/dd/yyyy.

Initial date (ngày ban đầu): Ngày mặc định được hiển thị khi Date Entry được tạo lần đầu tiên.

Max date (ngày tối đa): Ngày tối đa mà người dùng có thể chọn.

Min date (ngày tối thiểu): Ngày tối thiểu mà người dùng có thể chọn.

4.2. Ví dụ

Lưu ý: Để chạy được đoạn code bên dưới cần cài gói tkcalendar

```
import tkinter as tk
from tkinter import ttk
from tkinter import messagebox
from tkcalendar import DateEntry

def get_date():
    date = cal.get_date()
    messagebox.showinfo("Thông báo", f"Ngày được chọn: {date}")
```

```

root = tk.Tk()
root.title("Ví dụ Date Entry")

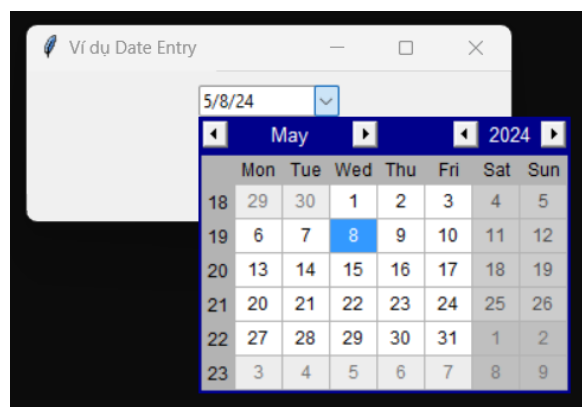
cal = DateEntry(root, width=12, background='darkblue',
foreground='white', borderwidth=2)
cal.pack(padx=10, pady=10)

button = tk.Button(root, text="Lấy ngày",
command=get_date)
button.pack(pady=5)

root.mainloop()

```

Kết quả



5. TreeView

TreeView trong Tkinter là một thành phần giao diện người dùng cho phép hiển thị dữ liệu dưới dạng cấu trúc cây, nơi mỗi mục có thể chứa các mục con. Thành phần này thường được sử dụng để hiển thị dữ liệu phân cấp, như dữ liệu danh mục, thư mục hoặc bảng dữ liệu có cấu trúc.

5.1. Các tùy chọn:

TreeView trong Tkinter có một số tùy chọn để tùy chỉnh và điều chỉnh cách hiển thị dữ liệu. Một số tùy chọn phổ biến bao gồm:

- **Columns (cột):** Cho phép định nghĩa các cột dữ liệu trong TreeView, để hiển thị thông tin chi tiết của mỗi mục.

- Header (tiêu đề): Cho phép hiển thị tiêu đề cho các cột trong TreeView.
- Select mode (chế độ lựa chọn): Xác định cách người dùng có thể lựa chọn mục trong TreeView, bao gồm single (chọn một mục duy nhất), browse (duyệt qua các mục), và extended (chọn nhiều mục).
- Styles (kiểu): Cho phép tùy chỉnh giao diện của các mục trong TreeView, bao gồm màu sắc, font chữ, ...
- Events (sự kiện): Cung cấp các sự kiện để xử lý khi người dùng tương tác với TreeView, như khi chọn một mục, mở rộng hoặc thu gọn một nút, ...

5.2. Ví dụ

```
import tkinter as tk
from tkinter import ttk

root = tk.Tk()
root.title("Ví dụ TreeView")

# Tạo TreeView
tree = ttk.Treeview(root)
tree["columns"] = ("Name", "Age")

# Đặt tên cho các cột
tree.column("#0", width=100, minwidth=100, stretch=tk.NO)
tree.column("Name", width=100, minwidth=100, stretch=tk.NO)
tree.column("Age", width=100, minwidth=100, stretch=tk.NO)

# Thêm tiêu đề cho các cột
tree.heading("#0", text="ID", anchor=tk.W)
tree.heading("Name", text="Name", anchor=tk.W)
tree.heading("Age", text="Age", anchor=tk.W)

# Thêm dữ liệu vào TreeView
```

```

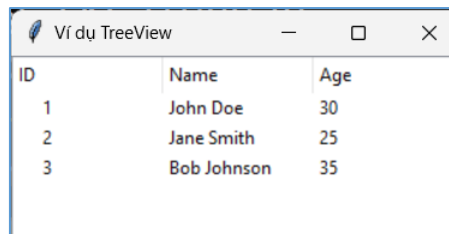
tree.insert("", "end", text="1", values=("John Doe",
"30"))
tree.insert("", "end", text="2", values=("Jane Smith",
"25"))
tree.insert("", "end", text="3", values=("Bob Johnson",
"35"))

tree.pack()

root.mainloop()

```

Kết quả



ID	Name	Age
1	John Doe	30
2	Jane Smith	25
3	Bob Johnson	35

5.3. Ví dụ 2

```

import tkinter as tk
from tkinter import ttk

def add_child():
    selected_item = tree.focus()
    if selected_item:
        tree.insert(selected_item, "end", text="New
Child", values=("Child Value"))

root = tk.Tk()
root.title("Treeview Example")

# Create Treeview

```

```
tree = ttk.Treeview(root)
tree["columns"] = ("Value")

# Define columns
tree.column("#0", width=150, minwidth=100, stretch=tk.NO)
tree.column("Value", width=100, minwidth=100,
stretch=tk.NO)

# Headings
tree.heading("#0", text="Parent", anchor=tk.W)
tree.heading("Value", text="Value", anchor=tk.W)

# Add data
parent1 = tree.insert("", "end", text="Parent 1",
values=("Value 1"))
tree.insert(parent1, "end", text="Child 1", values=("Child
Value 1"))
tree.insert(parent1, "end", text="Child 2", values=("Child
Value 2"))

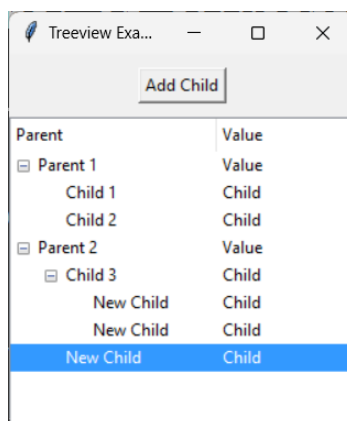
parent2 = tree.insert("", "end", text="Parent 2",
values=("Value 2"))
tree.insert(parent2, "end", text="Child 3", values=("Child
Value 3"))

# Buttons
add_button = tk.Button(root, text="Add Child",
command=add_child)
add_button.pack(pady=10)
```

```
tree.pack()

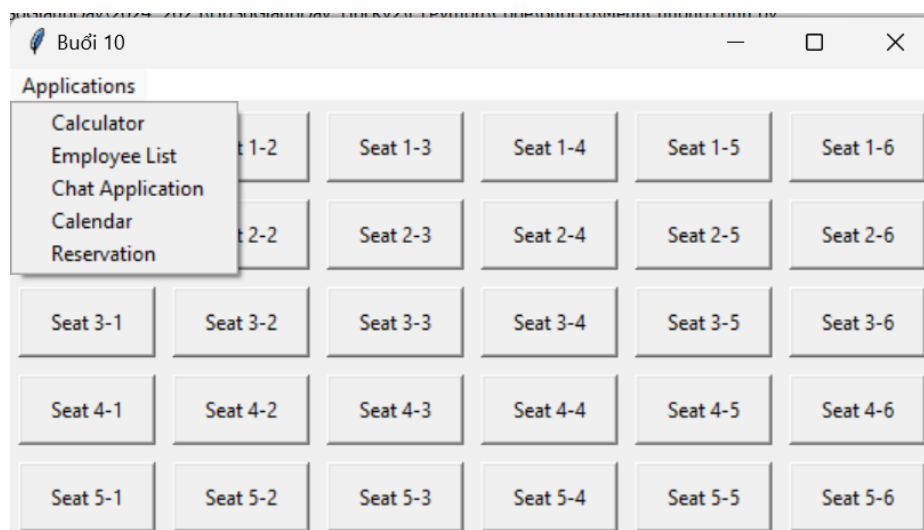
root.mainloop()
```

Kết quả



BÀI TẬP TRÊN LỚP PHẦN 2

1. Tạo menu để thực thi tất cả các bài tập trong phần 1



1. Ứng dụng Ghi chú đơn giản:

- Tạo một cửa sổ Tkinter với một ô văn bản để người dùng nhập nội dung ghi chú.
- Thêm một menu có các lệnh như "Lưu", "Mở", "Thoát" để thực hiện các chức năng tương ứng.
- Cho phép người dùng chọn màu sắc cho văn bản ghi chú bằng Colorchooser.

Yêu cầu chức năng

- **Cửa sổ chính**
 - Tiêu đề: "Simple Note"
 - Kích thước cố định (ví dụ 600×400), nền trắng.

- **Text widget**
 - Chiếm toàn bộ vùng nội dung, cho phép nhập/sửa văn bản.
- **Menu bar**
 - **File**
 - *Lưu* (Save): mở dialog chọn đường dẫn, lưu nội dung Text ra file .txt.
 - *Mở* (Open): mở dialog chọn file .txt, đọc và hiển thị lên Text.
 - *Thoát* (Exit): đóng ứng dụng.
 - **Format**
 - *Chọn màu chữ* (Color...): mở Colorchooser, áp dụng màu cho toàn bộ Text.
- **Colorchooser**
 - Cho phép chọn màu chữ.
 - Cập nhật ngay Text widget.
- **Xử lý lỗi**
 - Khi lưu/mở thất bại, hiển thị `messagebox.showerror`.

2. Quản lý Thông tin Sinh viên:

- Tạo một cửa sổ Tkinter với một Treeview để hiển thị thông tin sinh viên.
- Thêm các cột trong Treeview để hiển thị thông tin như tên, tuổi, giới tính, lớp, điểm,...
- Sử dụng menu để thêm, sửa, xóa thông tin sinh viên.
- Cho phép người dùng nhập ngày sinh của sinh viên bằng Date Entry.

Yêu cầu chức năng

- **Cửa sổ chính**
 - Tiêu đề: “Student Manager”
 - Kích thước: ~800×500.
- **Treeview** (ttk.Treeview)
 - Cột: “Mã SV”, “Họ tên”, “Tuổi”, “Giới tính”, “Lớp”, “Ngày sinh”, “Điểm TB”
 - Cho phép kéo thanh cuộn dọc.
- **Menu bar**
 - **Action**
 - *Thêm sinh viên* (Add): mở dialog/form nhập liệu.
 - *Sửa sinh viên* (Edit): chọn dòng, sửa thông tin trên form.
 - *Xóa sinh viên* (Delete): xóa dòng đang chọn, xác nhận trước.
 - *Thoát* (Exit).
- **Form thêm/sửa**
 - Entry cho Mã SV, Họ tên, Tuổi, Lớp, Điểm TB.
 - Combobox hoặc Radiobutton cho Giới tính.
 - DateEntry (từ tkcalendar) cho Ngày sinh.
 - Nút “Lưu” / “Hủy”.

- **Lưu/đọc dữ liệu**
 - Từ file CSV/JSON (tuỳ chọn).
- **Xác thực dữ liệu**
 - Tuổi và Điểm TB phải là số hợp lệ, Ngày sinh không vượt quá ngày hiện tại.

3. Ứng dụng Quản lý Sự kiện:

- Tạo một cửa sổ Tkinter với một Treeview để hiển thị danh sách sự kiện.
- Thêm các cột trong Treeview để hiển thị thông tin như tên sự kiện, ngày diễn ra, địa điểm,...
- Sử dụng menu để thêm, sửa, xóa sự kiện.
- Cho phép người dùng chọn màu sắc cho từng sự kiện bằng Colorchooser.

Yêu cầu chức năng

- **Cửa sổ chính**
 - Tiêu đề: “Event Manager”
 - Kích thước ~800×500.
- **Treeview**
 - Cột: “ID sự kiện”, “Tên sự kiện”, “Ngày diễn ra”, “Địa điểm”, “Màu sắc”.
- **Menu bar / Toolbar**
 - *Thêm sự kiện* (Add Event)
 - *Sửa sự kiện* (Edit Event)
 - *Xóa sự kiện* (Delete Event)
 - *Thoát*
- **Form thêm/sửa**
 - Entry cho ID, Tên sự kiện, Địa điểm.
 - DateEntry cho Ngày diễn ra.
 - Colorchooser cho Màu sắc (hiển thị preview ô vuông màu).
 - Nút “Lưu” / “Huỷ”.
- **Lưu/đọc**
 - File JSON hoặc SQLite đơn giản.
- **Hiển thị màu**
 - Cột “Màu sắc” hiển thị ô vuông hoặc tên màu.

4. Ứng dụng Ghi chú và Lập kế hoạch:

- Tạo một cửa sổ Tkinter với hai Tab hoặc Frame: một để ghi chú và một để lập kế hoạch.
- Trong Tab Ghi chú, người dùng có thể nhập nội dung ghi chú và chọn màu sắc bằng Colorchooser.
- Trong Tab Lập kế hoạch, người dùng có thể thêm, sửa, xóa các sự kiện trong lịch sử và chọn ngày bằng Date Entry.

Yêu cầu chức năng

- **Cửa sổ chính**
 - Tiêu đề: “Note & Planner”

- Kích thước ~900×600.
- **Tab control** (ttk.Notebook) hoặc hai Frame chuyển đổi
 - **Tab “Ghi chú”**
 - Text widget lớn để nhập ghi chú.
 - Nút/ menu “Chọn màu” (Colorchooser) cho Text.
 - Nút “Lưu ghi chú”, “Mở ghi chú”.
 - **Tab “Lập kế hoạch”**
 - Treeview hiển thị danh sách kế hoạch với các cột: “Sự kiện”, “Ngày”, “Mô tả”.
 - Nút “Thêm”, “Sửa”, “Xóa” kế hoạch.
 - Form thêm/sửa có Entry cho sự kiện, Text cho mô tả, DateEntry cho ngày.
 - Lưu/đọc file JSON.
- **Chuyển tab mượt**
 - Tab “Ghi chú” ↔ “Lập kế hoạch”.
- **Lưu trạng thái**
 - Khi mở lại, giữ nguyên tab đang chọn và nội dung.

7. Bài tập về nhà

Bài tập 1: Ứng dụng Tạo Form Tự Động

- **Mục tiêu:** Xây dựng ứng dụng cho phép người dùng tự do thêm các trường nhập liệu (Entry, Checkbutton, Radiobutton, ...) vào một form.

Yêu cầu chức năng

1. Thêm trường

- Người dùng chọn loại widget (Entry, Checkbutton, Radiobutton, Text, Combobox...) và nhập **label** cho trường.
- Khi “Thêm”, widget mới được tạo và hiển thị lên form.

2. Xóa trường

- Danh sách các trường hiện có hiển thị dưới dạng Listbox hoặc Treeview; người dùng chọn và nhấn “Xóa” để gỡ khỏi form.

3. Lưu form

- Nút “Lưu form” cho phép:
 - Xuất cấu trúc form ra **text** (dạng mô tả: loại widget, label, vị trí layout).
 - Hoặc lưu thành file **JSON** chứa danh sách trường với thông tin {
 "type": "...", "label": "...", "options":
 [...], "layout": {...} }.

Yêu cầu giao diện/UI

- Thanh công cụ hoặc khung bên phải để chọn loại widget và nhập label.
- Khu vực chính hiển thị form động (canvas hoặc frame) dùng grid hoặc pack.
- Danh sách trường hiện có (Listbox/Treeview) hiển thị ở khung bên dưới.
- Các nút “Thêm”, “Xóa”, “Lưu form” đặt rõ ràng, có tooltip mô tả.

Yêu cầu kỹ thuật bổ sung

- Sử dụng OOP: mỗi trường là một đối tượng, lưu trữ loại, label, và tham chiếu đến widget.
- Khi layout thay đổi (thêm/xóa), cập nhật lại positions tự động.
- Bắt lỗi nhập label trống, hủy khi người dùng nhấn “Hủy” hoặc “Đóng”.

Tiêu chí đánh giá

- Đầy đủ chức năng thêm/xóa/lưu.
- Form tự cân chỉnh layout khi thay đổi.
- File JSON đầu ra hợp lệ, có cấu trúc rõ ràng.

Bài tập 2: Ứng dụng Cửa Sổ Nhiều Màn Hình với Menu

- **Mục tiêu:** Tạo một ứng dụng với cửa sổ chính có menubar và các lệnh mở ra các cửa sổ phụ (Toplevel) cho các chức năng khác nhau.
- **Yêu cầu:**
 - Cửa sổ chính có menubar với các mục như “Settings”, “Help”, “About”.
 - Mỗi lệnh trong menu mở ra một cửa sổ Toplevel riêng biệt, trong đó bạn cần áp dụng các layout khác nhau (ví dụ: sử dụng grid cho cửa sổ Settings, pack cho cửa sổ About).
 - Thực hiện chức năng đóng/mở các cửa sổ một cách hợp lý.

Bài tập 3: Ứng dụng Vẽ và Chọn Màu

- **Mục tiêu:** Xây dựng một bảng vẽ đơn giản sử dụng widget Canvas kết hợp với Colorchooser để người dùng chọn màu vẽ.
- **Yêu cầu:**
 - Sử dụng Canvas để tạo khung vẽ, cho phép người dùng vẽ bằng chuột (ví dụ: vẽ đường thẳng, hình tròn).
 - Thêm nút “Chọn màu” sử dụng Colorchooser để cập nhật màu cọ vẽ.
 - Sắp xếp các nút chức năng bằng layout phù hợp (ví dụ: pack ở bên cạnh hoặc grid ở dưới Canvas).

Bài tập 4: Ứng dụng Hẹn Giờ và Đếm Ngược

- **Mục tiêu:** Tạo một ứng dụng đếm ngược, cho phép người dùng nhập thời gian kết thúc thông qua giao diện (có thể dùng DateEntry hoặc các widget nhập giờ).
- **Yêu cầu:**
 - Sử dụng DateEntry (hoặc các Entry kèm Label) để cho phép nhập ngày giờ.
 - Hiển thị thời gian còn lại bằng một Label, cập nhật theo thời gian thực (sử dụng hàm after của Tkinter).
 - Các nút “Bắt đầu”, “Tạm dừng” và “Reset” được bố trí hợp lý bằng layout grid.

Bài tập 5: Ứng dụng Quản Lý Tập Tin (File Explorer) với TreeView

- **Mục tiêu:** Xây dựng giao diện khám phá tập tin (File Explorer) sử dụng TreeView để hiển thị cấu trúc thư mục và tập tin.
- **Yêu cầu:**

- Duyệt thư mục hệ thống (hoặc một thư mục mẫu) và hiển thị tên thư mục, tập tin dưới dạng cây phân cấp.
- Sử dụng menu để cung cấp các lệnh như “Mở”, “Đổi tên”, “Xóa” (các lệnh này có thể chỉ là giả lập).
- Áp dụng layout grid để chia không gian giữa TreeView và vùng chi tiết (nếu có).

Bài tập 6: Ứng dụng Xem Ảnh với Chức Năng Zoom – Pan

- **Mục tiêu:** Tạo một ứng dụng xem ảnh đơn giản sử dụng Canvas, cho phép người dùng mở ảnh, zoom in/zoom out và pan (di chuyển ảnh).
- **Yêu cầu:**
 - Sử dụng Menu để thêm các chức năng “Mở ảnh”, “Zoom in”, “Zoom out”, “Reset”.
 - Bố trí Canvas và các nút điều khiển bằng layout grid hoặc pack.
 - Tích hợp xử lý sự kiện chuột cho chức năng di chuyển (pan) ảnh.

Bài tập 7: Máy Tính với Lịch Sử Tính Toán (Calculator with History)

- **Mục tiêu:** Mở rộng ứng dụng máy tính cơ bản bằng cách ghi lại lịch sử các phép tính trong một TreeView.
- **Yêu cầu:**
 - Thiết kế giao diện máy tính với các nút số và phép tính, sắp xếp bằng grid.
 - Mỗi khi thực hiện phép tính, lưu lại công thức và kết quả trong một TreeView hiển thị lịch sử tính toán.
 - Cho phép người dùng click vào một kết quả lịch sử để tự động đưa lại công thức vào ô nhập.

Bài tập 8: Ứng dụng Quản Lý Chi Tiêu Cá Nhân

- **Mục tiêu:** Xây dựng ứng dụng theo dõi chi tiêu với giao diện nhập liệu và hiển thị dữ liệu dưới dạng danh sách.
- **Yêu cầu:**
 - Sử dụng DateEntry để nhập ngày giao dịch, Entry để nhập số tiền và mô tả.
 - Hiển thị danh sách giao dịch trong một TreeView với các cột: Ngày, Mô tả, Số tiền.
 - Thêm menu với các tùy chọn “Thêm”, “Sửa”, “Xóa” giao dịch.
 - Áp dụng kết hợp layout grid và pack để bố trí giao diện hợp lý.

Bài tập 9: Ứng dụng Ghi Chú và Quản Lý Công Việc Đa Tab

- **Mục tiêu:** Tạo ứng dụng đa chức năng gồm ít nhất 2 tab với Notebook của ttk: một tab cho ghi chú, tab còn lại cho quản lý công việc.
- **Yêu cầu:**
 - Tab Ghi chú: Chứa Text widget cho nhập nội dung, nút “Chọn màu nền” sử dụng Colorchooser để thay đổi màu Text.

- Tab Quản lý công việc: Hiển thị danh sách công việc trong TreeView (với các cột như “Công việc”, “Trạng thái”), kèm các nút thêm – xóa công việc.
- Bố trí chung các tab theo layout rõ ràng và thân thiện với người dùng.

Bài tập 10: Dashboard Tương Tác Linh Hoạt

- **Mục tiêu:** Thiết kế giao diện dashboard tích hợp nhiều widget (Label, Button, Entry, TreeView, ...) với khả năng chuyển đổi giữa các kiểu layout (Pack, Grid, Place) thông qua menu.
- **Yêu cầu:**
 - Tạo giao diện chứa nhiều khu vực: khu vực thông tin, khu vực thống kê (có thể sử dụng TreeView hoặc Canvas vẽ đồ thị đơn giản).
 - Thêm một menu cho phép người dùng chuyển đổi giữa các kiểu bố trí (ví dụ: “Layout Pack”, “Layout Grid”, “Layout Place”) và tự động điều chỉnh giao diện khi chọn.
 - Tích hợp chức năng “Chọn màu chủ đề” dùng Colorchooser để thay đổi giao diện dashboard.